Tessera: Fast Join-Aggregates over Skewed Sparse Relations in Database Engines

https://arxiv.org/abs/2610.40620

1\. Problem

- 대규모 Sparse Relation에서 JOIN 후 COUNT/SUM 수행 시 중간 JOIN 결과가 과도하게 커지는 문제 해결 목적
- 데이터의 Skewness와 Sparsity로 인해 기존 JOIN 및 Matrix-based 방식의 연산 비용 증가
- 실제 결과는 작지만 중간 데이터 생성 및 처리 비용이 커지는 것이 핵심 병목
- 중요성: 불필요한 중간 결과 생성을 제거해 JOIN-AGGREGATE 성능 개선 가능

2\. Method

- Tessera: Skew-aware Join-Aggregate Operator 제안
- 핵심 아이디어: Sparse/Dense 데이터 특성에 따라 연산 경로를 분리하고 중간 JOIN 결과 생성을 최소화

\- Direct Sparse Matrix Product

- 원본 데이터를 기반으로 희소 행렬 곱 직접 수행
- 대규모 JOIN 중간 결과 생성 방지
- Non-zero 결과만 출력

\- Threshold-based Pruning

- 임계값을 만족할 가능성이 없는 항목을 사전에 제거
- 불필요한 곱셈 및 연산 감소

\- Dense-block Acceleration

- 빈도가 높은 Dense 영역을 별도로 처리

- 조밀한 영역은 CPU Matrix Accelerator(AMX)로 오프로드

\- 기존 방법과 차이

- JOIN 중간 결과 생성 대신 직접 집계
- Sparse/Dense 영역별 차별화된 실행
- Threshold 기반 사전 pruning 적용
- 하드웨어 가속과 알고리즘을 함께 최적화

3\. Experiments & Results

\- 실험 환경

- DuckDB, PostgreSQL, SQLite, Spark
- NYTimes, PubMed, MovieLens, Kosarak, Gene, LiveJournal, Orkut 등
- 16-core Intel Xeon 환경

\- 주요 결과

- 기존 최적 시스템 대비 2.8–18.8× 성능 향상
- DuckDB 대비 최대 1,802× 성능 향상
- Gene 데이터: 30분 → 1초
- 1 thread → 16 threads에서 8–12× 성능 향상
- NYTimes 데이터:
  - Tessera: 2.9초
  - SPALM: 54.3초
  - DuckDB: 8.6분
- Top-k 쿼리:
  - Tessera: 1초 미만
  - DuckDB: 226초

\- 세부 결과

- Item Pruning: 최대 2.93× 개선
- Grouped Pair Loop: 1.40–1.96× 개선
- Pruning + Accelerator: 최대 5.53× 개선

- Matrix Accelerator: 최대 6.88× 추가 가속

4\. Insight

\- 핵심 기여

- Sparse와 Dense 데이터 특성을 동시에 고려해 JOIN-AGGREGATE 실행 구조 자체를 최적화한 것

\- 한계점

- Intel AMX 중심 구현으로 다른 하드웨어 환경에 대한 검증 부족
- Dense 영역이 충분하지 않은 경우 Matrix Accelerator 효과 제한적
- 특정 JOIN-AGGREGATE 쿼리 패턴에 최적화된 구조
- 데이터 특성에 따라 기존 DB 실행 계획보다 성능이 낮아질 가능성 존재

\- 가져갈 포인트

- Data-aware Optimization: 데이터의 Sparsity와 Skewness를 함께 고려하는 것이 중요
- Early Pruning: 연산을 빠르게 하는 것보다 불필요한 연산 자체를 제거하는 것이 효과적일 수 있음
- Adaptive Execution: 데이터 특성에 따라 Sparse/Dense 실행 경로를 분리하는 접근
- Hardware-Algorithm Co-design: 하드웨어 가속과 알고리즘을 함께 설계해 성능 극대화